Московский государственный технический университет
Факультет ИУ «Информатика и системы управления»
Кафедра ИУ-1 «Системы автоматического управления»

ОТЧЕТ

по семинару №3; №4

по дисциплине

«Микропроцессорные и встраиваемые системы»

Выполнил: Зонов А.С.
Фуртат М.А.
Салженикин Д.А.
Галиев Э.Р.
Группа: ИУ1-52Б

Проверил: Ермаков И.Ю.

Работа выполнена: 02.12.2023

Отчет сдан:
Оценка:

2023 г.

Содержание

Цель работы 3
Ход работы 3
Вывод 10

Цель работы

Цель работы: приобретение практических навыков программирования на языке С и их отработка в среде разработки STM32.

Ход работы

Изменить значения напряжения (делителя) так, чтобы его собственное значение стало равным 0 либо значение D1, либо D2. Заполнить таблицу ниже и построить графики (Рисунок 1):

делитель, V	D1	D2
2,6	3073	3073
2,5	2938	2942
2,4	2835	2830
2,3	2643	2647
2,2	2547	2544
2,1	2410	2415
2	2303	2304
1,9	2198	2194
1,8	2043	2039
1,7	1908	1911
1,6	1726	1729
1,5	1607	1610
1,4	1479	1482
1,3	1358	1362
1,2	1217	1220
1,1	1082	1080
1	925	924
0,9	777	778
0,8	700	699
0,7	545	546
0,6	382	383
0,5	299	298
0,4	152	152
0,3	6	5
0,2	0	0
0,1	0	0
0	0	0

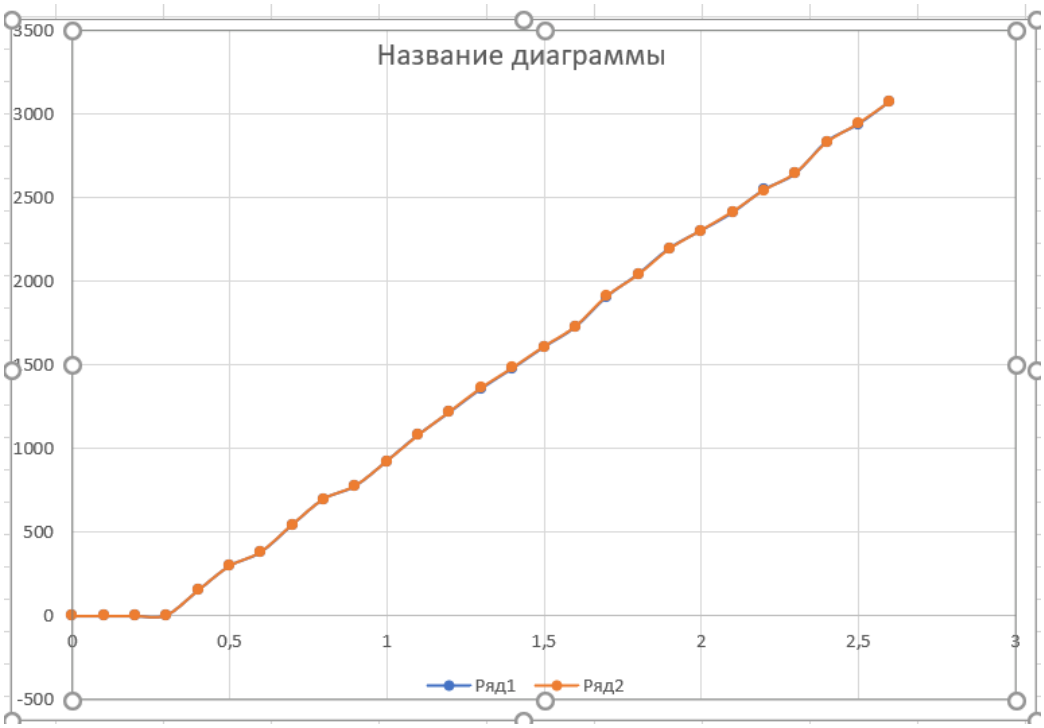


Рисунок 1

Код программы:

```
#include "main.h"

#include "stm32l476xx.h"

extern uint32_t systemTimeCounter;
uint32_t *spi2_dr = 0x40003c00 + 0x0C;

void delayMs(uint32_t delay)
{
	uint32_t curr = systemTimeCounter;
	while(systemTimeCounter - curr < delay)
	{	}
}



int main(void)
{
	// PWR AND SYS ENABLE
	RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;
	RCC->APB1ENR1 |= RCC_APB1ENR1_PWREN;
	RCC->CR |= RCC_CR_MSIRGSEL;
	SysTick_Config(4000000/1000);
	NVIC_SetPriority(SysTick_IRQn,0);
	NVIC_EnableIRQ(SysTick_IRQn);


	//ADC
	RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;
	GPIOA->ASCR |= GPIO_ASCR_ASC6 | GPIO_ASCR_ASC7;
```

```
      RCC->AHB2ENR |= RCC_AHB2ENR_ADCEN;
      RCC->CCIPR |= (3 << RCC_CCIPR_ADCSEL_Pos);

      ADC123_COMMON->CCR |= (3 << ADC_CCR_CKMODE_Pos);

      // calibration
      ADC1->CR &= ~(ADC_CR_DEEPPWD);
      ADC1->CR |= ADC_CR_ADVREGEN;
      ADC1->CR &= ~ADC_CR_ADEN;
      ADC1->CR &= ~ADC_CR_ADCALDIF;
      ADC1->CR |= ADC_CR_ADCAL;
      while(!(ADC1->CR & ADC_CR_ADCAL));

      // chanels cfg
      ADC1->SMPR2 |= (7 << ADC_SMPR2_SMP11_Pos) | (0 <<
ADC_SMPR2_SMP12_Pos);
      ADC1->SQR1 |= (1 << ADC_SQR1_L_Pos) | (11 << ADC_SQR1_SQ1_Pos) |
(12 << ADC_SQR1_SQ2_Pos);
      ADC1->CFGR &= (3 << ADC_CFGR_RES_Pos);
      ADC1->CFGR |= ADC_CFGR_CONT;

      ADC1->ISR |= ADC_ISR_ADRDY;
      ADC1->CR |= ADC_CR_ADEN;
      while(!(ADC1->ISR & ADC_ISR_ADRDY));

//


   // Led
      RCC->AHB2ENR |= RCC_AHB2ENR_GPIOBEN;
      GPIOB->MODER &=  ~(3<< GPIO_MODER_MODE2_Pos);
      GPIOB->MODER |= 1 << GPIO_MODER_MODE2_Pos;

      GPIOB->ODR |= GPIO_ODR_OD2;



  while (1)
  {
        ADC1->CR |= ADC_CR_ADSTART;
        while(!(ADC1->ISR & ADC_ISR_EOC));
        uint16_t d1 = ADC1->DR;
        ADC1->ISR |= ADC_ISR_EOC;
        while(!(ADC1->ISR & ADC_ISR_EOC));
        uint16_t d2 = ADC1->DR;
        ADC1->ISR |= ADC_ISR_EOC;
        delayMs(1000);
        GPIOB->ODR ^= GPIO_ODR_OD2;

  }
}
```

Task 1, добавить в таблицу ниже значения частот, которые соответствуют каждому значению константы и построить графики (Рисунок 2)

константа	частота, Hz
0	48
1	97
2	194
3	387
4	485
5	970
6	1940
7	3810
8	2250
9	1610
10	4260
11	2710

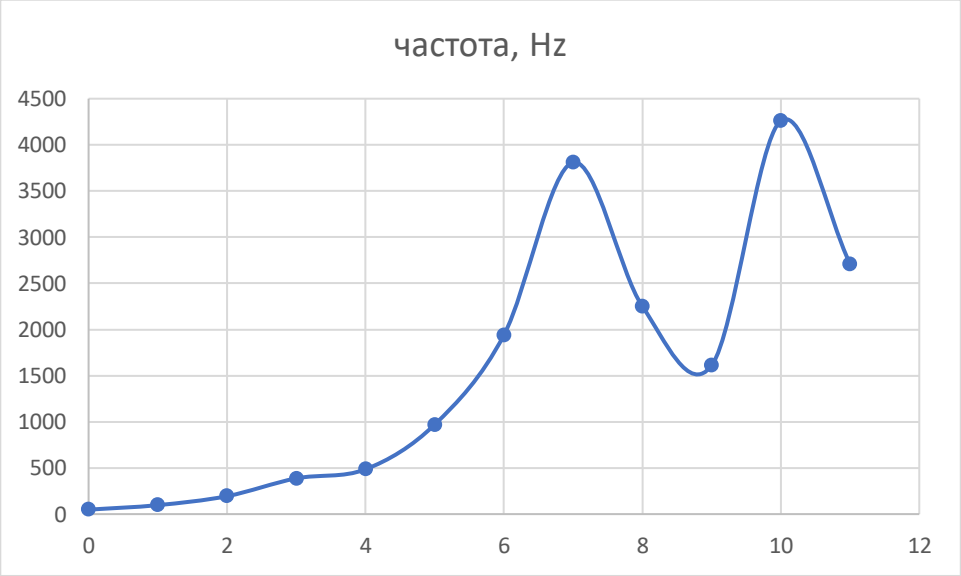


Рисунок 2

Task 2, добавить в таблицу ниже значения частот, которые соответствуют значениям констант, пока значение константы <255
и построить графики (Рисунок 3):

константа	частота, Hz
0	48.55
13	54
26	62
39	72

52	89
65	119
78	185
91	29
104	30
117	32
130	33,6
143	35,5
156	37,8
169	40,6
182	44
195	48,2
208	48.55
221	54
234	62
247	72

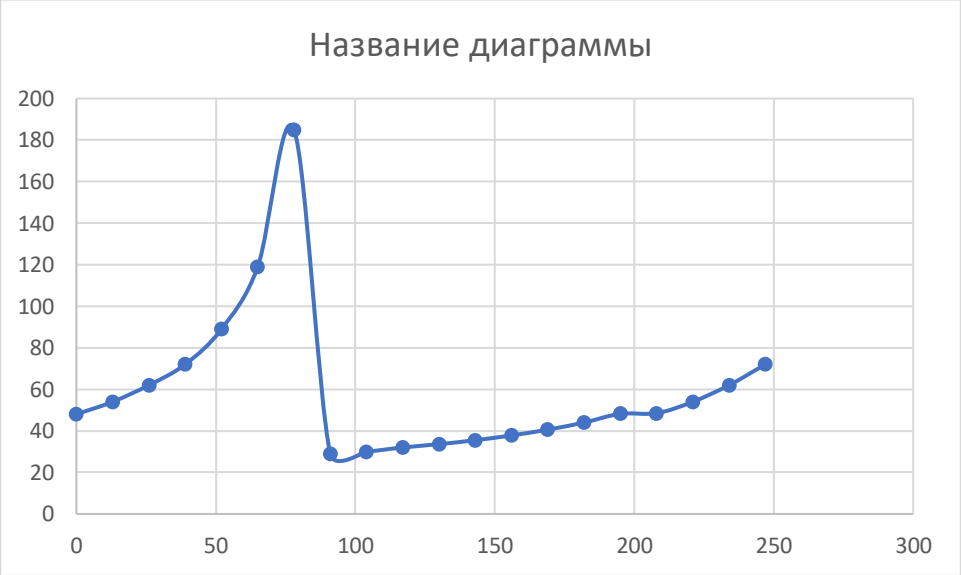


Рисунок 3

Task 3, добавить в таблицу ниже значения частот и констант, пока значение частоты не начнет повторяться, построить графики (Рисунок 4):

константа	частота, Hz
0	3680
1	3750
2	3440
3	3810
4	3960
5	3330

6	3750
7	4010
8	3930



Рисунок 4

Код программы:

```
#include "main.h"

#include "stm32l476xx.h"

extern uint32_t systemTimeCounter;
uint32_t *spi2_dr = 0x40003c00 + 0x0C;

void delayMs(uint32_t delay)
{
	uint32_t curr = systemTimeCounter;
	while(systemTimeCounter - curr < delay)
	{	}
}

uint8_t valueMsi = 0;
uint8_t valueCalibrateMsi = 0;
uint8_t valueCalibrateHsi = 0;
uint32_t kostil = 0;
void EXTI0_IRQHandler()
{
	EXTI->PR1 |= EXTI_PR1_PIF0;
	kostil++;
	if(kostil %2 != 0)
		return;
	NVIC_DisableIRQ(EXTI0_IRQn);
/*
	// task1

	valueMsi++;
```

```
	if(valueMsi >= 8)
	{
		FLASH->ACR &= ~FLASH_ACR_LATENCY;
		FLASH->ACR |= FLASH_ACR_LATENCY_1WS;
	}
	if(valueMsi>= 10)
	{
		FLASH->ACR &= ~FLASH_ACR_LATENCY;
		FLASH->ACR |= FLASH_ACR_LATENCY_2WS;
	}
	RCC->CR &= ~(RCC_CR_MSIRANGE);
	RCC->CR |= (valueMsi << RCC_CR_MSIRANGE_Pos);
	if(valueMsi == 11)
		valueMsi = -1;//*/
/*
	// task2

	RCC->ICSCR &= ~RCC_ICSCR_MSITRIM_Msk;
	RCC->ICSCR |= (valueCalibrateMsi << RCC_ICSCR_MSITRIM_Pos);
	valueCalibrateMsi+=13;//*/

	//task3

	RCC->ICSCR &= ~RCC_ICSCR_HSITRIM_Msk;
	RCC->ICSCR |= (valueCalibrateHsi << RCC_ICSCR_HSITRIM_Pos);
	valueCalibrateHsi+=1;
	if(valueCalibrateHsi > 127)
		valueCalibrateHsi = 0;//*/

	NVIC_EnableIRQ(EXTI0_IRQn);

}

int main(void)
{
	// PWR AND SYS ENABLE
	RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;
	RCC->APB1ENR1 |= RCC_APB1ENR1_PWREN;
	while(!(RCC->CR & RCC_CR_MSIRDY));
	RCC->CR &= ~(RCC_CR_MSIRANGE);
	RCC->CR |= RCC_CR_MSIRGSEL;

	// task3
	RCC->CR |= RCC_CR_HSION;
	while(!(RCC->CR & RCC_CR_HSIRDY));
	FLASH->ACR |= FLASH_ACR_LATENCY_2WS;
	RCC->CFGR |= RCC_CFGR_HPRE_DIV64;
	RCC->CFGR |= RCC_CFGR_SW_HSI;
	while(!(RCC->CFGR & RCC_CFGR_SWS_HSI));
		FLASH->ACR |= FLASH_ACR_LATENCY_2WS;
		RCC->CFGR |= RCC_CFGR_HPRE_DIV64;//*/

	RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

	GPIOA->MODER &= ~((3 << GPIO_MODER_MODE0_Pos)| (3
<<GPIO_MODER_MODE1_Pos ));
```

```
    GPIOA->MODER |= (1 << GPIO_MODER_MODE1_Pos);
    GPIOA->OSPEEDR |=  GPIO_OSPEEDER_OSPEEDR1;

    EXTI->IMR1 |= EXTI_IMR1_IM0;
    EXTI->RTSR1 |= EXTI_RTSR1_RT0;
    NVIC_SetPriority(EXTI0_IRQn,0);
    NVIC_EnableIRQ(EXTI0_IRQn);



while (1)
{

      GPIOA->ODR ^= GPIO_ODR_OD1;
      /*__asm("nop");
      __asm("nop");
     ...
                             __asm("nop");
                             __asm("nop");
                             //*/



}


 }
```

Вывод

В результате семинарской работы были приобретены практические навыки по программированию макетной платы на языке программирования С, а также выполнены задания в IDE STM32.